TECHNISCHES WHITEPAPER · 2026

‹ › Mesh.

Strukturelle Quellcode-Kompression für LLM-Kontexteffizienz — Mesh Compression Engine.

74%	**3,9×**	**$3,90**
Ø Token-Reduktion	Kontextgewinn im 128k-Fenster	vs $15,00/MTok · Claude Opus 4.6

DAS PROBLEM

Mit steigender Kontextauslastung fällt die LLM-Abrufgenauigkeit — messbar.

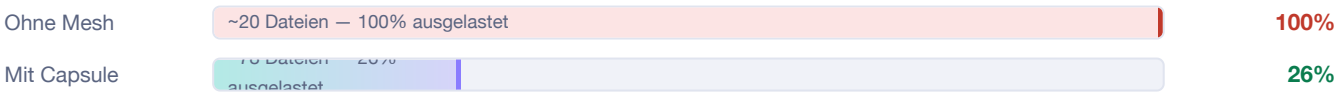
Large Language Models arbeiten mit einem fixen Kontextfenster in Tokens. Das ist keine weiche Grenze — sie erzeugt eine harte Genauigkeitsdecke, sobald die Auslastung Richtung 100 % tendiert.

NIAH — Needle In A Haystack

Standard-Retrieval-Benchmark (Liu et al., 2023, *„Lost in the Middle"*): Eine Zielinformation wird an verschiedenen Positionen in einem langen Dokument platziert; das Modell muss sie abrufen.

KONTEXTAUSLASTUNG	ABRUFGENAUIGKEIT
< 60 %	~95–98 %
60–80 %	~75–85 %
80–95 %	~55–70 %
> 95 %	~40–60 %

Die Degradation ist positionsabhängig: Informationen in der Mitte des Kontexts werden weniger zuverlässig abgerufen als solche am Anfang oder Ende. Bei Kontexten über 100k Tokens ist der Effekt verstärkt.



SWE-bench

Branchen-Benchmark für KI-Coding-Agenten bei echten GitHub-Issues. Spitzenmodelle (Claude Opus 4.6, GPT-4o, Gemini 3.1 Pro) lösen ca. **40–55 %** der Aufgaben (Stand Anfang 2026). Root-Cause-Analyse der Fehler identifiziert konsistent ein führendes Muster: **Der Agent hatte nicht genug Codebase-Kontext, um das Problem vollständig zu verstehen.**

Aktuell genutzte Tools wählen per Embedding-Retrieval relevante Dateien aus und senden sie roh ans Modell. Das bestimmt, was geladen wird — nicht, was jede Datei kostet. Je größer die Codebasis, desto geringer die Abdeckung pro Kontextfenster.

FUNKTIONSWEISE

Strukturelle Kompression mit selektiver Wiederherstellung — keine Zusammenfassung.

Capsule verarbeitet Quelldateien **bevor** sie ein LLM-Kontextfenster betreten. Die Originaldatei ist jederzeit vollständig wiederherstellbar.

Pipeline-Stufen

STUFE	OPERATION	OUTPUT
1 — Parse	Sprachbewusste AST-Extraktion	Strukturkomponenten: Deklarationen, Signaturen, Typ-Annotationen, Doc-Comments, Control-Flow-Marker
2 — Komprimieren	Implementierungsbody-Reduktion	Bodies durch strukturelle Stubs ersetzt; nicht-strukturelle Tokens minimiert
3 — Enkodieren	Deskriptor-Assembly	Kompakter Klartext-Deskriptor mit vollständigem semantischen Skelett
4 — Selektive Wiederherstellung	On-demand Body-Restaurierung	Vollständiger Implementierungsbody inline injiziert, wenn das Modell ihn anfordert

Was erhalten bleibt

- Alle Export- und Deklarationsnamen
- Alle Funktions- und Methodensignaturen (Parameter, Rückgabetypen)
- Typ-Definitionen, Interfaces und Typ-Aliase
- Klassenstruktur und Vererbungshierarchie
- Doc-Comment-Inhalt
- Control-Flow-Marker (try/catch-Präsenz, Schleifen, async/await)
- Modul-Imports und Abhängigkeitsreferenzen

Aktivierungsschwelle

Dateien unter ~200 Tokens erhalten Pass-through (Format-Overhead übersteigt die Einsparung). In Produktions-Codebasen liegen ~95 % der Dateien nach Token-Anzahl im Kompressionsbereich. Die Schwelle hat keinen materiellen Effekt auf die Gesamt-Kontextreduktion.

PERFORMANCE

Produktionspipeline · 5 Dateitypen · 6 Größenkategorien · 50 Dateien pro Kategorie

TypeScript, YAML, SQL, HTML, Markdown — reale Zahlen aus der laufenden Capsule-Pipeline.

DATEIGRÖSSE	TOKEN-REDUKTION	EINSPARUNG
~200B (xs) — wird übersprungen		+10%
~1 KB (klein)		–83%
~5 KB (mittel)		–95%
~18 KB (groß)		–98,5%
~50 KB (XL)		–99,4%
~100 KB (XXL)		–99,9%
Durchschnitt (klein–XXL)		–74% → 3,9×

Kontextfenster-Abdeckung bei 128k Tokens

Ohne Mesh	~20 Dateien — 100% ausgelastet	100%
Mit Capsule	~78 Dateien — 26% ausgelastet	26%

Warum die Kompressionsrate mit Dateigröße steigt

Implementierungsbodies skalieren mit der Dateigröße; strukturelle Skelette nicht. Eine 100-KB-TypeScript-Datei hat ungefähr dieselbe Skelettgröße wie eine 5-KB-Datei — die zusätzlichen 95 KB sind fast ausschließlich Implementierungsbodies. Die Kompressionsrate steigt daher asymptotisch mit wachsender Dateigröße.

WIRTSCHAFTLICHKEIT

74 % Kostenreduktion pro Query — gleiche Dateien, gleiches Modell, gleiche Ergebnisqualität.

Kosten pro Query (Claude Opus 4.6 — $15,00/MTok Input)

INPUT-TYP	GESENDETE TOKENS	KOSTEN PRO QUERY
Roher Code (100 mittlere Dateien)	1.000.000	$15,00
Capsule-komprimiert (dieselben 100 Dateien)	**~260.000**	**$3,90**

Skalierungsprojektion (10.000 Nutzer · 50 Queries/Tag · 50k Ø Kontext-Tokens)

METRIK	ROH	CAPSULE
Tokens gesamt/Tag	25.000.000.000	~6.500.000.000
API-Kosten/Tag (Claude Opus 4.6)	$375.000	**~$97.500**
Monatliche API-Kosten	$11.250.000	**~$2.925.000**
Monatliche Einsparung	**$8.325.000 (–74 %)**	

Die Kostenkurve ist mit Capsule flach — weitere Dateien hinzuzufügen kostet nahezu null Marginal-Tokens, sobald sie im Kompressionsbereich liegen.

SYSTEM-DESIGN

Retrieval wählt Dateien. Kompression reduziert, was jede Datei kostet.

```
User-Query
    |
    ▼
Workspace-Indexer        ← scannt Projektverzeichnis, baut Datei-Registry
    |
    ▼
File-Selektor            ← Embedding-basiertes Retrieval, gibt gerankte Kandidaten zurück
    |
    ▼
Capsule-Kompression      ← strukturelle Kompression pro Datei (parallele Worker)
    |
    ▼
Context-Assembler        ← packt Deskriptoren, zielt auf ~65 % Auslastung
    |
    ▼
Model-API-Call           ← Claude / Gemini / GPT (provider-agnostisch)
    |
    ▼
Selektive Recovery       ← stellt vollständige Bodies auf Modellanfrage wieder her
    |
    ▼
Response + Diff-Output
```

Retrieval und Kompression — nicht gegenseitig ausschließend

EIGENSCHAFT	EMBEDDING-RETRIEVAL	CAPSULE-KOMPRESSION
Was das Modell erreicht	Roher Dateiinhalt (volle Token-Kosten)	**Komprimierter Deskriptor**
Token-Kosten skalieren mit	Dateianzahl × Dateigröße	**Nahezu konstant (Skelettgröße)**
Kontextabdeckung (128k)	~20 mittlere Dateien	**~78 mittlere Dateien**
Codebase-Größen-Sensitivität	Hoch	**Niedrig — Rate steigt mit Größe**
Additiv mit Retrieval	N/A	**Ja — operiert nach dem Retrieval**

Modell-Kompatibilität

Capsule-Output ist Klartext in einem strukturierten Format. Kein Fine-Tuning, kein spezieller Tokenizer. Kompatibel mit jedem transformer-basierten LLM: Claude, Gemini, GPT-4o, o1, o3, o4-mini. Provider-Wechsel ist zur Laufzeit konfigurierbar — kein Re-Indexing erforderlich.

Dateityp-Unterstützung (v1)

SPRACHE	BEWAHRTE STRUKTURELLE ELEMENTE
TypeScript / JavaScript	Exports, Klassen-Deklarationen, Funktionssignaturen, Typ-Definitionen, Interfaces, Generics
Python	Module-level-Deklarationen, Klassen-/Funktionssignaturen, Type Hints, Docstrings
YAML / JSON	Top-Level-Keys, Schema-Struktur (Werte auf Typ-Marker komprimiert)
SQL	Tabellen-/View-/Funktionsdefinitionen, Spaltennamen und -typen, Index-Definitionen
HTML / JSX	Komponentenbaumstruktur, Prop-Signaturen, Slot-Struktur
Markdown	Überschriftenhierarchie, Code-Block-Präsenz, Link-Struktur

Go, Rust, Java, C#, Ruby, PHP geplant für Capsule v2.